

Linguaggi Formali e Compilatori

(Formal Languages and Compilers)

prof. S. Crespi Reghizzi, prof. Angelo Morzenti
(prof. Luca Breveglieri)

Prova scritta - 2 ottobre 2012 - Parte I: Teoria

CON SOLUZIONI - A SCOPO DIDATTICO LE SOLUZIONI SONO MOLTO ESTESE E COMMENTATE VARIAMENTE - NON SI RICHIEDE CHE IL CANDIDATO SVOLGA IL COMPITO IN MODO ALTRETTANTO AMPIO, BENSÌ CHE RISPONDA IN MODO APPROPRIATO E A SUO GIUDIZIO RAGIONEVOLE

NOME:

MATRICOLA:

FIRMA:

ISTRUZIONI - LEGGERE CON ATTENZIONE:

- L'esame si compone di due parti:
 - I (80%) Teoria:
 1. espressioni regolari e automi finiti
 2. grammatiche libere e automi a pila
 3. analisi sintattica e parsificatori
 4. traduzione sintattica e analisi semantica
 - II (20%) Esercitazioni Flex e Bison
- Per superare l'esame l'allievo deve sostenere con successo entrambe le parti (I e II), in un solo appello oppure in appelli diversi, ma entro un anno.
- Per superare la parte I (teoria) occorre dimostrare di possedere conoscenza sufficiente di tutte le quattro sezioni (1-4), rispondendo alle domande obbligatorie.
- È permesso consultare libri e appunti personali.
- Per scrivere si utilizzi lo spazio libero e se occorre anche il tergo del foglio; è vietato allegare nuovi fogli o sostituirne di esistenti.
- Tempo: Parte I (teoria): 2h.15m - Parte II (esercitazioni): 60m

1 Espressioni regolari e automi finiti 20%

1. I linguaggi L' e L'' sono definiti tramite le espressioni regolari R' e R'' seguenti:

$$R' = \left((a b \mid b)^* a \right)^*$$

$$R'' = \left(b (b a \mid a a)^* \right)^+$$

Si risponda alle domande seguenti:

- (a) Si elenchino le stringhe di lunghezza ≤ 3 appartenenti a ciascuno dei linguaggi.
 - (b) Si costruisca un automa finito A che riconosce il linguaggio definito dall'intersezione $L' \cap L''$ e si descriva a parole il metodo seguito per costruirlo.
 - (c) (facoltativa) Se necessario si trasformi l'automa A in quello deterministico e minimo B equivalente ad A .
-

Soluzione

- (a) Ecco le stringhe di lunghezza ≤ 3 appartenenti a ciascuno dei linguaggi:

$$L' \ni \varepsilon, a, aa, ba, aaa, aba, baa, bba$$

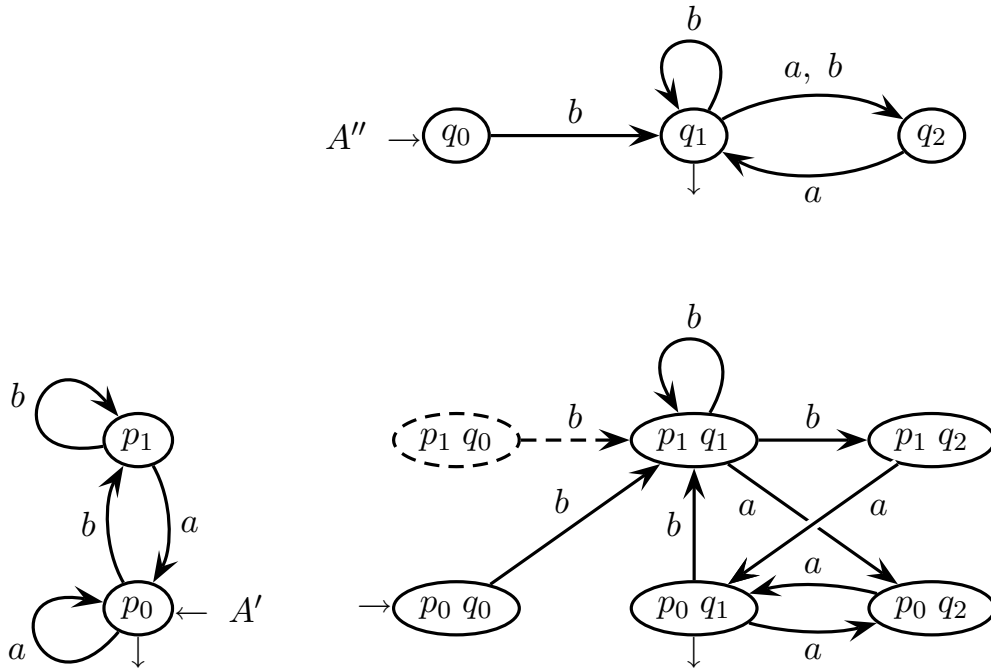
Si noti che il linguaggio L' contiene la stringa vuota e tutte e sole quelle terminanti con a .

$$L'' \ni b, bb, baa, bba, bbb$$

Il linguaggio L'' non contiene la stringa vuota ε .

- (b) Con uno dei metodi noti (Thompson o Berry-Sethi) o intuitivamente, prima si costruiscono gli automi A' e A'' che riconoscono i due linguaggi L' e L'' (qui intuitivamente). Poi per mezzo della costruzione nota come prodotto cartesiano, si costruisce la macchina che riconosce il linguaggio intersezione $L' \cap L''$.

Poiché tale macchina prodotto può avere un numero di stati pari al prodotto degli stati delle macchine fattore, per ridurre il lavoro si farà attenzione a semplificarle il più possibile (stati e archi tratteggiati sono irraggiungibili o indefiniti e si possono eliminare). Ecco un automa che riconosce il linguaggio $L' \cap L''$:



L'automa fattore A' è deterministico e minimo, mentre l'automa fattore A'' non è deterministico, però è in forma ridotta (tutti gli stati sono raggiungibili e definiti). L'automa prodotto cartesiano è pure in forma ridotta.

- (c) La macchina del prodotto cartesiano non è deterministica. Si determinizza ed eventualmente minimizza con le note costruzioni, lasciate al lettore.

2 Grammatiche libere e automi a pila 20%

1. Si pensi al seguente processo produttore-consumatore. Il produttore p mette *un* elemento in un buffer e il consumatore c toglie *un* elemento dal buffer.

Il primo linguaggio L_1 di alfabeto $\{c, p\}$ contiene le sequenze tali che

nel buffer non si verifica mai underflow, e (1)

all'inizio il buffer è vuoto (ma non necessariamente alla fine) (2)

Esempi:

$\varepsilon, pc, ppc, pcpc, ppcpc, \dots$

Al contrario sono errate per esempio le stringhe: cp e $pccp$.

Invece il secondo linguaggio L_2 di alfabeto $\{c, d, p\}$ modella un processo dove ci sono due tipi di consumatore: c come nel caso precedente e d che toglie *due* elementi. Il linguaggio L_2 di alfabeto $\{c, d, p\}$ soddisfa entrambe le condizioni (1) e (2).

Ecco i nuovi esempi:

$\varepsilon, ppd, pcppd, ppcpd, pppdcp, \dots$

Al contrario sono errate per esempio le stringhe: pcd , pd e pdp .

Si risponda alle domande seguenti:

- (a) Si progetti una grammatica G_1 non estesa che genera il linguaggio L_1 .
 - (b) Si modifichi la grammatica G_1 per ottenere una grammatica G_2 non estesa che genera il linguaggio L_2 .
 - (c) (facoltativa) Se necessario, si trovino versioni G'_1 e G'_2 non ambigue delle grammatiche G_1 e G_2 costruite ai punti precedenti.
-

Soluzione

- (a) C'è un solo consumatore di tipo c . Il linguaggio L_1 somiglia a quello di Dyck assimilando p alla parentesi aperta e c a quella chiusa, e ne differisce solo perché la frase può terminare senza “chiudere tutte le parentesi aperte”.

Si trova subito una grammatica G_1 facile ma ambigua. Eccola (assioma S):

$$G_1 \left\{ \begin{array}{l} S \rightarrow \varepsilon \\ S \rightarrow p S c S \\ S \rightarrow p S S \end{array} \right. \quad \left| \quad \begin{array}{c} \begin{array}{c} S \\ / \quad \backslash \\ p \quad S \quad S \\ / \quad | \quad \backslash \\ p \quad S \quad S \quad \varepsilon \\ | \quad | \\ \varepsilon \quad \varepsilon \end{array} \quad \begin{array}{c} S \\ / \quad \backslash \\ p \quad S \quad S \\ | \quad | \quad \backslash \\ \varepsilon \quad p \quad S \quad S \\ | \quad | \\ \varepsilon \quad \varepsilon \end{array} \end{array}$$

Si tratta della regola di Dyck replicata con e senza lettera c . Questa versione di G_1 è fortemente ambigua: già la stringa corta pp ha due alberi sintattici. Si migliora semplificando la regola $S \rightarrow p S S$ in quella equivalente $S \rightarrow p S$, così:

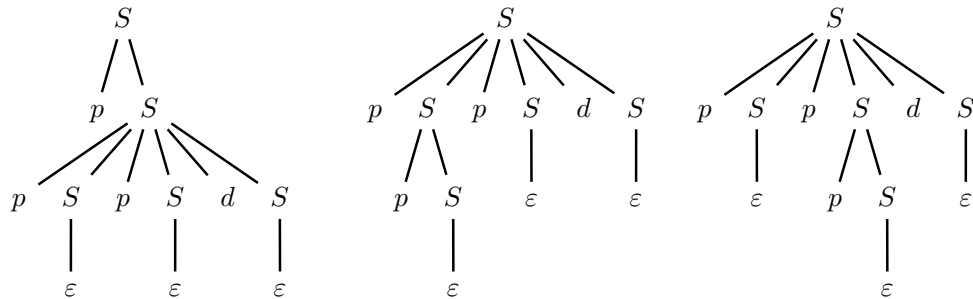
$$G_1 \left\{ \begin{array}{l} S \rightarrow \varepsilon \\ S \rightarrow p S c S \\ S \rightarrow p S \end{array} \right. \quad \left| \quad \begin{array}{c} \begin{array}{c} S \\ / \quad \backslash \\ p \quad S \\ / \quad | \quad \backslash \\ p \quad S \quad c \quad S \\ | \quad | \\ \varepsilon \quad \varepsilon \end{array} \quad \begin{array}{c} S \\ / \quad \backslash \\ p \quad S \quad c \quad S \\ / \quad | \quad \backslash \\ p \quad S \quad \varepsilon \end{array} \end{array}$$

Però anche questa versione è ambigua: la stringa ppc ha due alberi sintattici.

- (b) Si prenda ora il linguaggio L_2 con due tipi di consumatore. La facile versione ambigua della grammatica G_2 è questa (assioma S), semplificando come prima e mettendo in evidenza le regole dei due consumatori:

$$G_2 \left\{ \begin{array}{l} S \rightarrow \varepsilon \mid p S \\ S \rightarrow p S c S \quad - \text{consumatore } c \\ S \rightarrow p S p S d S \quad - \text{consumatore } d \end{array} \right.$$

Si tratta ancora della regola di Dyck con lettere p raddoppiate per estenderla al caso di consumatore d . Questa versione di G_2 è perfino più ambigua di G_1 : già la stringa corta $pppd$ ha tre alberi sintattici.



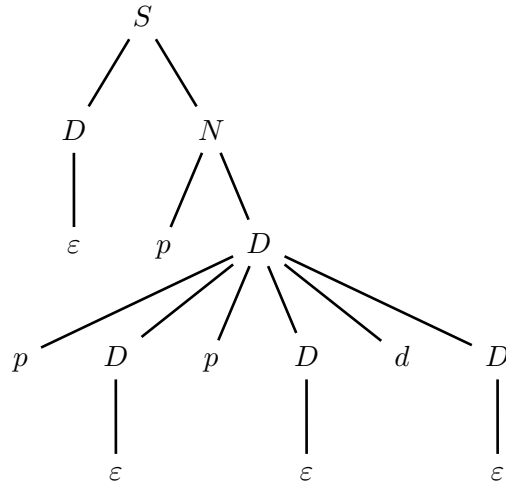
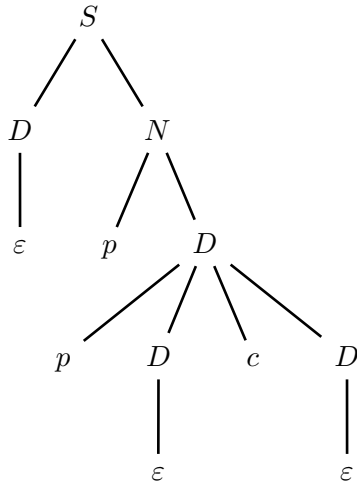
- (c) Per disambiguare si può ragionare così: la stringa consiste di una stringa bilanciata ossia di Dyck, eventualmente seguita da una o più stringhe di Dyck con almeno una lettera p in eccesso. Ecco il caso (a) (assioma S):

$$G'_1 \left\{ \begin{array}{l} S \rightarrow D N \mid D \\ D \rightarrow p D c D \mid \varepsilon \\ N \rightarrow p D N \mid p D \end{array} \right.$$

Ora la stringa ppc ha un solo albero sintattico. Ecco il caso (b) (assioma S):

$$G'_2 \left\{ \begin{array}{l} S \rightarrow D N \mid D \\ D \rightarrow p D p D d D \mid p D c D \mid \varepsilon \\ N \rightarrow p D N \mid p D \end{array} \right.$$

Valgono le osservazioni su G'_1 e la stringa $pppd$ ha un solo albero sintattico.



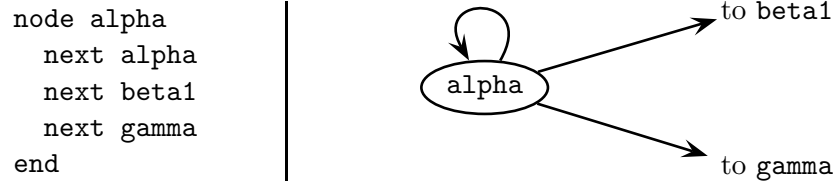
2. Si consideri un linguaggio per descrivere un grafo orientato con nodi etichettati, un solo nodo iniziale e uno solo finale. Il linguaggio ha queste funzioni:

- (a) gli identificatori sono alfanumerici e si indicano con quest'unico terminale: `id`
- (b) i nodi del grafo si dichiarano così:

```
nodelist alpha, N2, beta1, gamma end
```

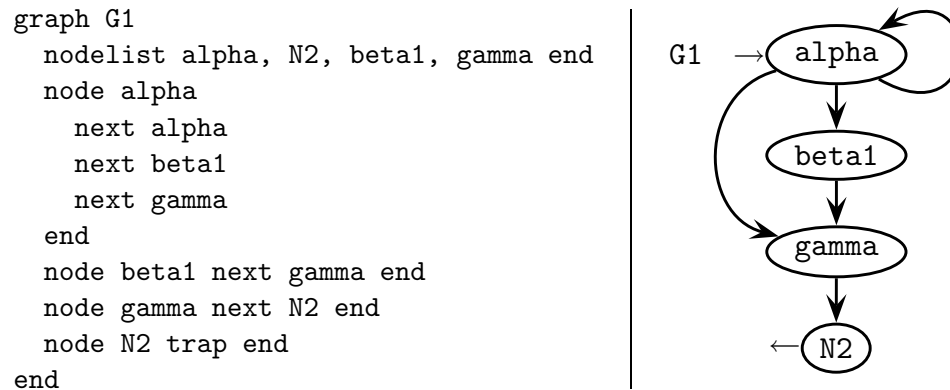
i primi due sono l'iniziale e il finale obbligatori, poi quelli interni facoltativi

- (c) gli archi uscenti da ciascun nodo e gli autoanelli si definiscono così:

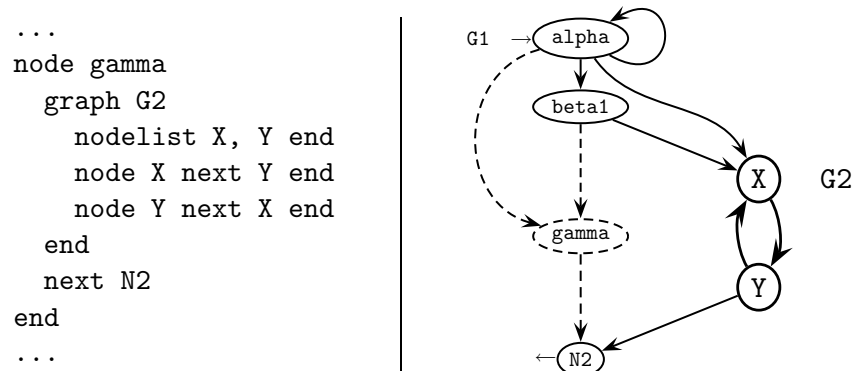


un nodo può non avere archi uscenti, per esempio: `node N2 trap end`

- (d) esempio di grafo con quattro nodi:



- (e) un nodo si può espandere ricorsivamente in un grafo, per esempio così:



si intende che gli archi entranti in `gamma` arrivano a `X`, nodo iniziale di `G2`, e quelli uscenti da `gamma` partono da `Y`, nodo finale di `G2` (che qui non ha nodi interni)

Si scriva una grammatica estesa (EBNF) per generare il linguaggio così tratteggiato. Si indichino in breve gli aspetti semantici non modellabili sintatticamente.

Soluzione

Ecco la grammatica estesa non ambigua richiesta (assioma GRAPH):

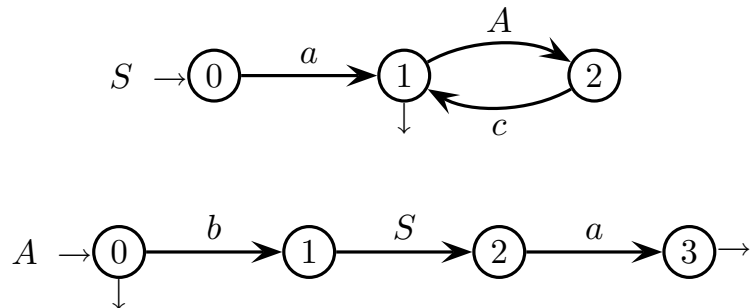
$$\left\{ \begin{array}{l} \langle \text{GRAPH} \rangle \rightarrow \text{graph } \langle \text{ID} \rangle \langle \text{G_BODY} \rangle \text{ end} \\ \langle \text{ID} \rangle \rightarrow \text{id} \\ \langle \text{G_BODY} \rangle \rightarrow \langle \text{N_DEC} \rangle \langle \text{N_LIST} \rangle \\ \langle \text{N_DEC} \rangle \rightarrow \text{nodelist } \langle \text{ID} \rangle (, \langle \text{ID} \rangle)^+ \text{ end} \\ \langle \text{N_LIST} \rangle \rightarrow [\langle \text{NODE} \rangle]_2^* \\ \langle \text{NODE} \rangle \rightarrow \text{node } \langle \text{ID} \rangle [\langle \text{GRAPH} \rangle] \langle \text{N_BODY} \rangle \text{ end} \\ \langle \text{N_BODY} \rangle \rightarrow \langle \text{X_LIST} \rangle \mid \text{trap} \\ \langle \text{X_LIST} \rangle \rightarrow (\langle \text{NEXT} \rangle)^+ \\ \langle \text{NEXT} \rangle \rightarrow \text{next } \langle \text{ID} \rangle \end{array} \right.$$

Le parentesi quadre senza indici indicano opzionalità, e quelle con pedice e apice indicano ripetizione da un numero minimo di volte (pedice) a uno massimo (apice). Sono operatori regolari di tipo esteso e volendo facilmente riducibili a quelli standard. La grammatica non è ambigua in quanto è costruita aggregando componenti e strutture notoriamente non ambigui.

Non si può modellare sintatticamente la concordanza di nome tra dichiarazione e descrizione di un nodo, ossia tra le classi sintattiche N_DEC e N_LIST.

3 Analisi sintattica e parsificatori 20%

1. Si consideri la grammatica estesa seguente, data come rete di macchine ricorsive di alfabeti terminale $\{ a, b \}$ e nonterminale $\{ S, A \}$ (assioma S):



Si risponda alle domande seguenti (con la metodologia di analisi estesa *ELR* e *ELL*):

- (a) Si costruisca il grafo pilota della grammatica e si spieghi se essa è di tipo *ELR*(1).
- (b) Si spieghi se la grammatica è di tipo *ELL*(1).
- (c) (facoltativa) Si effettui l'analisi *ELR* della stringa di esempio seguente:

$a b a a c$

Si usi la tabella predisposta di seguito (numero di righe non significativo).

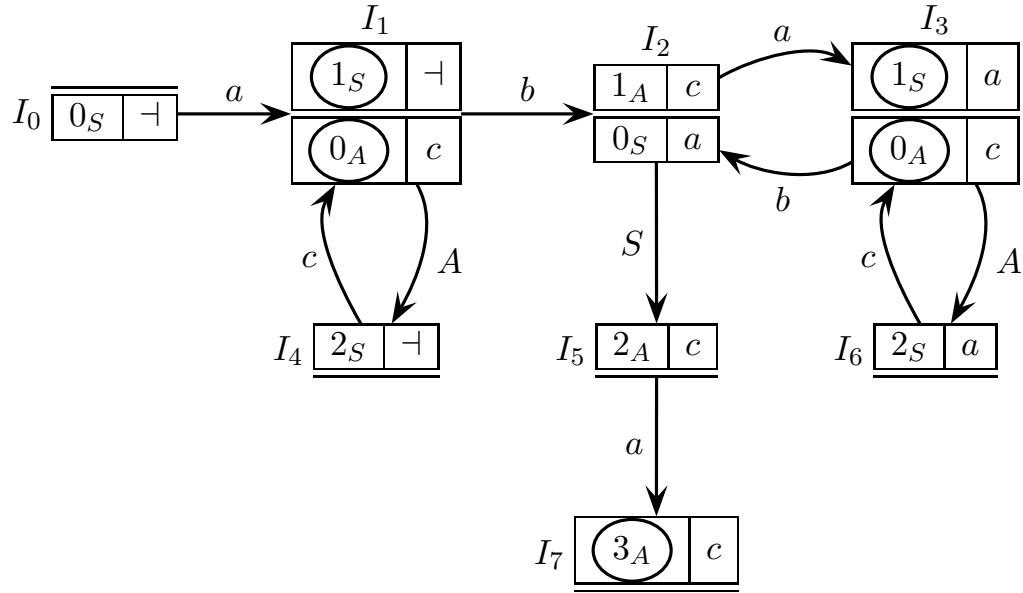
Chi preferisse procedere con la metodologia classica di analisi *LR* e *LL*, usi la grammatica non estesa seguente (assioma S) equivalente alla rete di macchine:

$$\left\{ \begin{array}{l} S \rightarrow a X \\ X \rightarrow A c X \mid \varepsilon \\ A \rightarrow b S a \mid \varepsilon \end{array} \right.$$

—	a	b	a	a	c	mossa
$0_S \perp$						

Soluzione

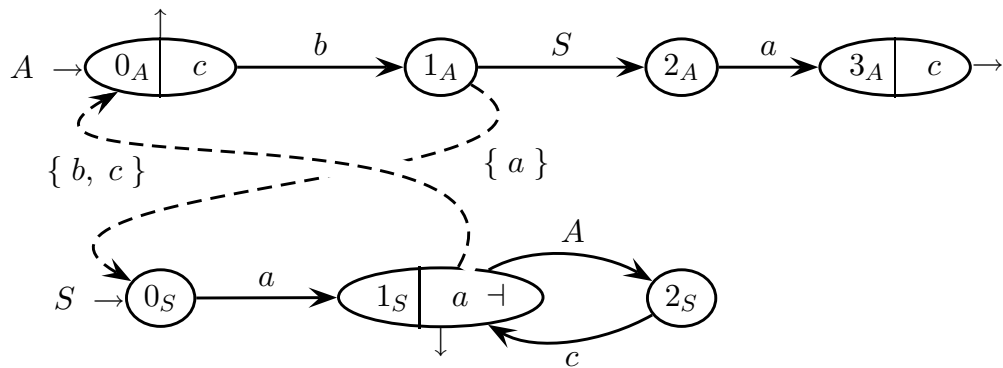
(a) Ecco il grafo pilota *ELR* della grammatica estesa:



Il grafo pilota non ha conflitti spostamento-riduzione o riduzione-riduzione. Pertanto la grammatica estesa è di tipo *ELR*(1).

(b) Il grafo pilota della grammatica soddisfa la condizione *ELR*, soddisfa la condizione di Beatty e la grammatica non ha ricorsioni sinistre. Pertanto la grammatica estesa è anche di tipo *ELL*(1).

Qui non occorre disegnare il grafo pilota *ELL* semplificato della grammatica. Comunque eccolo per completezza:



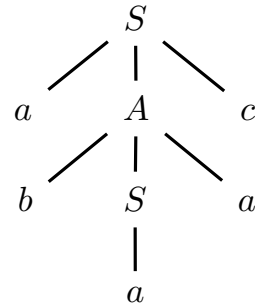
Naturalmente questo grafo risulta deterministico come atteso.

(c) Ecco la tabella di simulazione dell'analisi *ELR* per la stringa campione:

base	a	b	a	a	c	mossa
0 $0_S \perp$	$a \qquad b$		a	a	c	shift a
	1 1_S 0_A #1 $\perp$	b	a	a	c	shift b
	1 1_S 0_A #1 $\perp$	2 1_A 0_S #2 $\perp$	a	a	c	shift a
	1 1_S 0_A #1 $\perp$	2 1_A 0_S #2 $\perp$	3 1_S 0_A #2 $\perp$	a	c	$a \rightsquigarrow S$
	1 1_S 0_A #1 $\perp$	2 1_A 0_S #2 $\perp$	S	a	c	shift S
	1 1_S 0_A #1 $\perp$	2 1_A 0_S #2 $\perp$	5 2_A #1	a	c	shift a
	1 1_S 0_A #1 $\perp$	2 1_A 0_S #2 $\perp$	5 2_A #1	7 3_A #1	c	$b S a \rightsquigarrow A$
	1 1_S 0_A #1 $\perp$	A			c	shift A
	1 1_S 0_A #1 $\perp$	4 2_S #1			c	shift c
	1 1_S 0_A #1 $\perp$	4 2_S #1			1 1_S #1	$a A c \rightsquigarrow S$
S					accept	

In ciascun elemento della pila, il macrostato I_i è segnalato come [i] ed è seguito dalle candidate attive o iniziali con il puntatore rispettivo.

L'albero sintattico corrispondente è questo:



Per quanto riguarda la grammatica non estesa, essa risulta di tipo *LR*(1) e *LL*(1). L'analisi *LR* della stringa non presenta difficoltà. I dettagli sono sul libro di testo.

4 Traduzione e analisi semantica 20%

1. Sono dati questi digrammi di lettere e insiemi di lettere iniziali e finali, di alfabeto $\Sigma = \{ a, b, c \}$:

$$\text{Digrammi} = \left\{ \underbrace{ab}_1, \underbrace{bb}_2, \underbrace{bc}_3, \underbrace{cb}_4, \underbrace{ca}_5 \right\}$$

$$\text{Iniziali} = \{ a \} \quad \text{Finali} = \{ b \}$$

Si risponda alle domande seguenti:

- (a) Si consideri il linguaggio locale $L_1 \subset \Sigma^*$ definito dai digrammi, dagli inizi e dalle fini. Si disegni il traduttore finito T_1 , se possibile deterministico, per calcolare la traduzione τ_1 che ha il linguaggio L_1 come sorgente e che per ogni digramma emette in uscita il numero associato come specificato sopra. Ecco un esempio:

$$\tau_1 (a b b c a b) = 1 2 3 5 1$$

- (b) Si consideri il linguaggio libero $L_2 \subset \Sigma^*$ di tipo Dyck, dove le coppie di parentesi aperta-chiusa corrispondenti ammissibili sono i digrammi. Si scriva lo schema sintattico di traduzione G_2 (o la grammatica di traduzione) che ha il linguaggio L_2 come sorgente e che per ogni coppia di parentesi corrispondenti emette in uscita il numero associato come specificato sopra. Ecco un esempio:

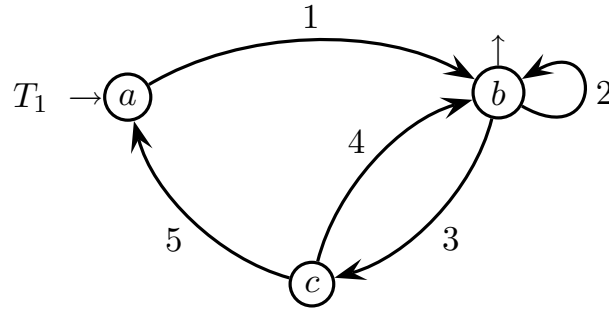
$$\tau_2 \left(\underbrace{a \underbrace{bc} b}_{\underbrace{\hspace{1.5cm}}} \underbrace{ca} \right) = 3 1 5$$

con questo allineamento: $\frac{a \quad b \quad c \quad b \quad c \quad a}{3 \quad 1 \quad 5}$

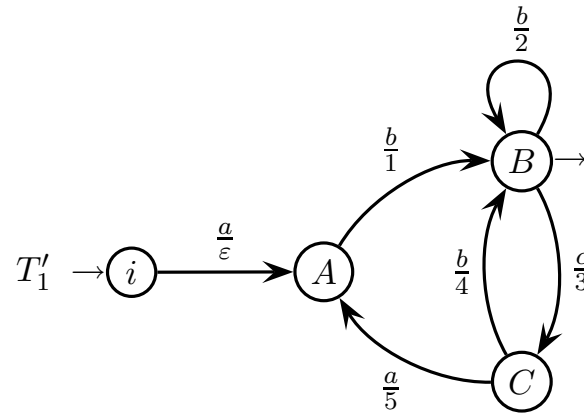
- (c) (facoltativa) Il linguaggio destinazione della traduzione τ_1 è locale? Si argomenti brevemente la risposta, per il sì o per il no. Poi si dia una descrizione informale (a parole) sintetica del linguaggio destinazione della traduzione τ_2 .

Soluzione

- (a) L'automa traduttore finito T_1 di τ_1 è locale (o meglio lo è l'automa riconoscente soggiacente) con nodi identificati dai simboli d'ingresso e con archi etichettati dai simboli di uscita, ossia i numeri dei digrammi corrispondenti agli archi. Eccolo:



Si mette il traduttore locale T_1 nella forma standard di traduttore finito spostando le lettere sorgente sugli archi in ingresso ai nodi e introducendo uno stato iniziale nuovo. Eccolo:



Il traduttore finito di τ_1 così ottenuto è ovviamente deterministico.

- (b) Lo schema sintattico G_2 è quello di Dyck, dove la parte d'uscita associa il numero alla parentesi chiusa. Eccolo (assioma S):

$$G_2 \left\{ \begin{array}{l|l} S \rightarrow \varepsilon & S \rightarrow \varepsilon \\ S \rightarrow a S b S & S \rightarrow S 1 S \\ S \rightarrow b S b S & S \rightarrow S 2 S \\ S \rightarrow b S c S & S \rightarrow S 3 S \\ S \rightarrow c S b S & S \rightarrow S 4 S \\ S \rightarrow c S a S & S \rightarrow S 5 S \end{array} \right.$$

- (c) Il linguaggio destinazione di τ_1 è locale. Infatti osservando l'automa traduttore T_1 si vede che ciascun arco è identificato da un numero. La connettività del grafo è data semplicemente elencando i digrammi degli archi entranti-uscenti in uno stesso nodo, ed elencando inizi e fini. Per completezza ecco la caratterizzazione locale del linguaggio di uscita:

$$\text{Digrammi} = \{ 12, 13, 22, 23, 34, 35, 42, 43, 51 \}$$

$$\text{Iniziali} = \{ 1 \} \quad \text{Finali} = \{ 1, 2, 4 \}$$

Pertanto il linguaggio destinazione di τ_2 è locale, sebbene sia più complesso di quello sorgente poiché ha nove digrammi e tre fini. Gioca ruolo essenziale non avere archi con la stessa etichetta, altrimenti non si potrebbe caratterizzare il linguaggio destinazione mediante digrammi, inizi e fini.

Il linguaggio destinazione di τ_2 è universale e basta guardare la grammatica destinazione G_2^d per capirlo. Eccola (a sinistra):

$$G_2^d \left\{ \begin{array}{l} S \rightarrow \varepsilon \\ S \rightarrow S 1 S \\ S \rightarrow S 2 S \\ S \rightarrow S 3 S \\ S \rightarrow S 4 S \\ S \rightarrow S 5 S \end{array} \right. \quad G_3^d \left\{ \begin{array}{l} S \rightarrow \varepsilon \\ S \rightarrow 1 S \\ S \rightarrow 2 S \\ S \rightarrow 3 S \\ S \rightarrow 4 S \\ S \rightarrow 5 S \end{array} \right.$$

Sostituendo la regola $S \rightarrow \varepsilon$ alla comparsa sinistra di S nelle altre regole, si ottiene una sotto-grammatica G_3^d (sopra a destra) che genera un sottinsieme del linguaggio destinazione di τ_2 , ossia si ha $L(G_3^d) \subseteq L(G_2^d)$.

La grammatica G_3^d è lineare a destra e palesemente genera il linguaggio universale. Infatti l'automa finito corrispondente ha un solo stato ed è proprio quello del linguaggio universale. Pertanto anche l'intero linguaggio destinazione di τ_2 è universale (e infatti come potrebbe mai essere più ampio di quello universale?).

2. Un conto corrente bancario matura interessi a tasso fisso annuale 5 % (si trascurano le tasse sugli interessi). Sono definite le quattro operazioni seguenti:

- apertura (**open**), prima operazione eseguibile, con due parametri: data d e importo iniziale versato a (amount)
- deposito (**deposit**) con due parametri: data d e importo versato a
- ritiro (**withdraw**) con due parametri: data d e importo ritirato a
- chiusura (**close**), ultima operazione eseguibile, con un solo parametro: data d

La data è espressa come numero di giorni dell'Era Volgare (che inizia il 1° giorno di 1 dC). Le operazioni si svolgono tutte in date differenti. La sequenza di operazioni su un conto viene generata da questa grammatica astratta (assioma ACCT):

$$\left\{ \begin{array}{l} \langle \text{ACCT} \rangle \rightarrow \text{open } d \ a \ \langle \text{OP_LIST} \rangle \\ \langle \text{OP_LIST} \rangle \rightarrow \text{deposit } d \ a \ \langle \text{OP_LIST} \rangle \\ \langle \text{OP_LIST} \rangle \rightarrow \text{withdraw } d \ a \ \langle \text{OP_LIST} \rangle \\ \langle \text{OP_LIST} \rangle \rightarrow \text{close } d \end{array} \right.$$

L'interesse totale viene liquidato chiudendo il conto e va calcolato sommando gli interessi parziali maturati negli intervalli di tempo tra un'operazione e la successiva.

Si consideri l'esempio seguente, una sequenza di operazioni su un conto:

- il conto viene aperto versando 3000 euro, il giorno ... diciamo 8000
- dopo 300 giorni vengono versati altri 2000 euro
- dopo altri 100 giorni vengono ritirati 1000 euro
- dopo ulteriori 120 giorni il conto viene chiuso

L'interesse totale ti va dunque calcolato così, dove $\tau = \frac{0,05}{365}$ è il tasso giornaliero:

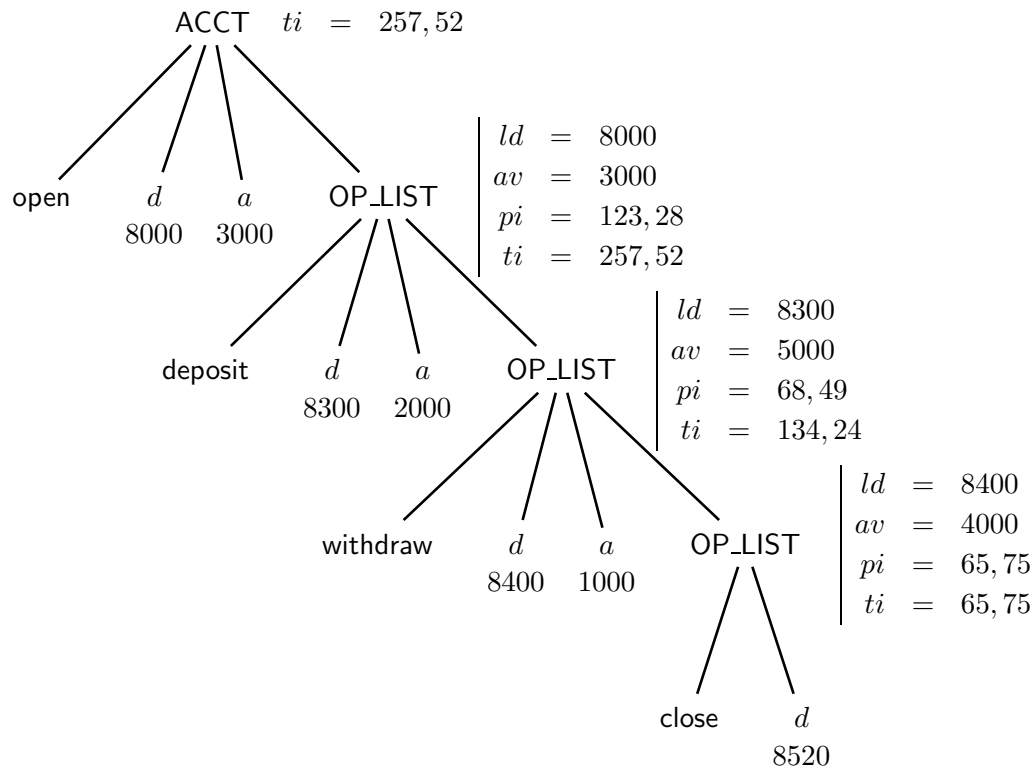
$$\begin{aligned} ti &= 3000 \times (8300 - 8000) \times \tau + 5000 \times (8400 - 8300) \times \tau \\ &\quad + 4000 \times (8520 - 8400) \times \tau \\ &= 123,28 + 68,49 + 65,75 = 257,52 \quad - \text{arrotondato per troncamento} \end{aligned}$$

Si risponda alle domande seguenti:

- (a) Si scriva una grammatica con attributi per calcolare l'interesse totale nella radice dell'albero delle operazioni sul conto. Gli attributi sono tutti dati, ma alcuni hanno specifica da completare. Le funzioni semantiche sono inferibili confrontando l'esempio di calcolo e l'albero decorato. Si vedano le pagine seguenti.
- (b) (facoltativa) Esaminando le dipendenze funzionali tra gli attributi, si stabilisca se l'interesse totale è calcolabile in una sola passata (one-sweep).

sx/dx	nome	(non)terminali	tipo	significato
	<i>ld</i>	OP_LIST	numero	<i>last date</i> , data dell'ultima operazione eseguita
	<i>av</i>	OP_LIST	numero	<i>account value</i> , valore cumulato da tutte le operazioni eseguite
	<i>pi</i>	OP_LIST	numero	<i>partial interest</i> , interesse parziale maturato tra l'ultima operazione e l'operazione corrente
	<i>ti</i>	ACCT, OP_LIST	numero	<i>total interest</i> , interesse totale maturato tra apertura e chiusura
dx	<i>d</i>	<i>d</i>	numero	<i>date</i> , restituisce la data dell'operazione corrente
dx	<i>a</i>	<i>a</i>	numero	<i>amount</i> , restituisce l'importo dell'operazione corrente

Gli attributi da usare sono questi e non ne occorrono altri, ma la specifica dei primi quattro (*ld*, *av*, *pi* e *ti*) va completata precisando se sono sinistri o destri. I due attributi in fondo sono specificati completamente e restituiscono i valori dei terminali associati (infatti si chiamano come quelli); convenzionalmente sono considerati destri.



Questo è l'albero decorato dell'esempio. Gli attributi sono scritti a destra del nonterminale di riferimento indipendentemente da che siano sinistri o destri.

$\langle \text{ACCT} \rangle_0 \rightarrow \text{open } d_1 \ a_2 \ \langle \text{OP_LIST} \rangle_3$	$ld =$ $av =$ $pi =$ $ti =$
$\langle \text{OP_LIST} \rangle_0 \rightarrow \text{deposit } d_1 \ a_2 \ \langle \text{OP_LIST} \rangle_3$	$ld =$ $av =$ $pi =$ $ti =$
$\langle \text{OP_LIST} \rangle_0 \rightarrow \text{withdraw } d_1 \ a_2 \ \langle \text{OP_LIST} \rangle_3$	$ld =$ $av =$ $pi =$ $ti =$
$\langle \text{OP_LIST} \rangle_0 \rightarrow \text{close } d_1$	$ld =$ $av =$ $pi =$ $ti =$

Qui vanno definite le funzioni semantiche che aggiornano gli attributi predisposti. Attenzione a mettere i pedici agli attributi, a sinistra e a destra dell'operatore di assegnamento $=$, coerentemente con le specifiche. *Non è detto che per ogni regola tutti gli attributi vadano calcolati poiché dipende se l'attributo è sinistro o destro.* Gli attributi d e a dei terminali in pratica sono funzioni e non occorre assegnare loro un valore, ciò avviene automaticamente. La costante τ è il tasso giornaliero d'interesse.

Soluzione

(a) Ecco gli attributi completati e le funzioni semantiche:

sx/dx	nome	(non)terminali	tipo	significato
dx	<i>ld</i>	OP_LIST	numero	<i>last date</i> , data dell'ultima operazione eseguita
dx	<i>av</i>	OP_LIST	numero	<i>account value</i> , valore cumulato da tutte le operazioni eseguite
sx	<i>pi</i>	OP_LIST	numero	<i>partial interest</i> , interesse parziale maturato tra l'ultima operazione e l'operazione corrente
sx	<i>ti</i>	ACCT, OP_LIST	numero	<i>total interest</i> , interesse totale maturato tra apertura e chiusura
dx	<i>d</i>	<i>d</i>	numero	<i>date</i> , restituisce la data dell'operazione corrente
dx	<i>a</i>	<i>a</i>	numero	<i>amount</i> , restituisce l'importo dell'operazione corrente

$\langle \text{ACCT} \rangle_0 \rightarrow \text{open } d_1 \ a_2 \ \langle \text{OP_LIST} \rangle_3$	$ld_3 = d_1$ $av_3 = a_2$ $pi = \text{non è calcolato qui}$ $ti_0 = ti_3$
$\langle \text{OP_LIST} \rangle_0 \rightarrow \text{deposit } d_1 \ a_2 \ \langle \text{OP_LIST} \rangle_3$	$ld_3 = d_1$ $av_3 = av_0 + a_2$ $pi_0 = av_0 \times (d_1 - ld_0) \times \tau$ $ti_0 = ti_3 + pi_0$
$\langle \text{OP_LIST} \rangle_0 \rightarrow \text{withdraw } d_1 \ a_2 \ \langle \text{OP_LIST} \rangle_3$	$ld_3 = d_1$ $av_3 = av_0 - a_2$ $pi_0 = av_0 \times (d_1 - ld_0) \times \tau$ $ti_0 = ti_3 + pi_0$
$\langle \text{OP_LIST} \rangle_0 \rightarrow \text{close } d_1$	$ld = \text{non è calcolato qui}$ $av = \text{non è calcolato qui}$ $pi_0 = av_0 \times (d_1 - ld_0) \times \tau$ $ti_0 = pi_0$

La grammatica con attributi è aciclica, dunque formalmente calcolabile. Si vede subito che produce l'albero decorato, dunque è ragionevolmente corretta.

- (b) È facile constatare che la grammatica con attributi proposta è a una passata (one-sweep). L'albero sintattico è lineare ossia in pratica una lista. Prima in discesa si calcolano e propagano gli attributi destri ld e av , che dipendono solo da attributi destri (compresi quelli associati ai terminali). Poi in salita si calcolano gli attributi sinistri pi e ti , che dipendono dagli attributi destri già calcolati e propagati oltre che da attributi sinistri. La condizione one-sweep è soddisfatta.